

Contents

Task 1.....	1
Task 2.....	2
Task 3.....	3
Task 4.....	4
Task 5.....	4
Task 6.....	5
Task 7.....	5
Task 8.....	7
Task 9.....	8
Task 10.....	8
Task 11.....	9
Task 12.....	9

Task 1

It enables the unit (service, timer and more) that even after the reboot process, it is started by systemd automatically. However, if we just run `systemctl enable`, it will not currently start the unit. If we want to start it without needing a reboot, we need to use `systemctl enable --now`. (BTW `sys` BELOW IS ALIAS FOR `systemctl` WHICH I DEFINED IN `/etc/bashrc`)

```
[root@rhel-9 ~ #] sys status httpd
o httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
  Active: inactive (dead)
  Docs: man:httpd.service(8)
[root@rhel-9 ~ #] sys enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
```

As you can see, httpd was not enabled in my system. I have enabled it. The link is created inside `/etc/systemd/system/multi-user.target.wants/httpd.service/<SERVICE_NAME>`.

Whenever we run enabling command, systemd basically generates a soft link from the unit file (which is inside `/usr/lib/systemd/system/`) into the proper `target's .wants/` or `.requires/` directory under `/etc/systemd/system/`.

Task 2

We can think it as more powerful variant of `systemctl disable` command. It basically blocks a unit, that is why we cannot enable or start it. The best example for it is `httpd` and `nginx`. They use the same ports. That is why if we try to start them both at the same time, the problem can occur.

We can use our mask in that case. Imagine in our application there is httpd running currently. So, we can block (mask) the `nginx`, so whenever anyone tries to run it, he cannot do it.

Let's show it. Initially, both are disabled and stopped:

```
[root@rhel-9 ~ #] systemctl status httpd
○ httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
  Active: inactive (dead)
  Docs: man:httpd.service(8)
[root@rhel-9 ~ #] systemctl status nginx
○ nginx.service - The nginx HTTP and reverse proxy server
  Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
  Active: inactive (dead)
```

Let's up httpd:

```
[root@rhel-9 ~ #] systemctl enable --now httpd #Both enable and start httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
```

Not let's try to up nginx (that causes a problem):

```
[root@rhel-9 ~ #] systemctl enable --now nginx
Created symlink /etc/systemd/system/multi-user.target.wants/nginx.service → /usr/lib/systemd/system/nginx.service.
Job for nginx.service failed because the control process exited with error code.
See "systemctl status nginx.service" and "journalctl -xeu nginx.service" for details.
```

So, we should mask `nginx`:

```
[root@rhel-9 ~ #] systemctl stop nginx
```

After that if we try to start `nginx`, a problem will occur:

```
[root@rhel-9 ~ #] systemctl start nginx
Failed to start nginx.service: Unit nginx.service is masked.
```

If we want to unmask and start to use `nginx`:

```
[root@rhel-9 ~ #] systemctl unmask nginx
Removed "/etc/systemd/system/nginx.service".
[root@rhel-9 ~ #] systemctl start nginx
[root@rhel-9 ~ #] systemctl is-active nginx
active
```

(In that case, we need to stop `httpd` beforehand)

Task 3

For `systemd` a unit is configuration file that tells our system what to manage. Basically, they are run by `systemd`.

There are a lot of types of units: services, timers, targets, sockets, mounts and more.

If we want to see all units including inactive ones, we use this command:

```
[root@rhel-9 ~ #] systemctl list-units --all
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
● boot.automount	not-found	inactive	dead	boot.automount
proc-sys-fs-binfmt_misc.automount	loaded	active	waiting	Arbitrarily
dev-cdrom.device	loaded	active	plugged	VMware VM
dev-disk-by\x2ddiskseq-1.device	loaded	active	plugged	VMware VM
dev-disk-by\x2ddiskseq-3.device	loaded	active	plugged	VMware VM

(If we want to see only active ones, we do not use `--all`).

If we want to see unit types (like service, socket and more), we need to run this command:

```
[root@rhel-9 ~ #] systemctl -t help
Available unit types:
service
mount
swap
socket
target
device
automount
timer
path
slice
scope
```

Task 4

We use this command to see all service units in the system:

```
[root@rhel-9 ~ #] systemctl list-units --type=service --all
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
accounts-daemon.service	loaded	inactive	dead	Accounts Service
alsa-restore.service	loaded	inactive	dead	Save/Restore Sound Card State
alsa-state.service	loaded	active	running	Manage Sound Card State (restore and store)
atd.service	loaded	active	running	Deferred execution scheduler
auditd.service	loaded	active	running	Security Auditing Service
● auto-cpufreq.service	not-found	inactive	dead	auto-cpufreq.service
● autofs.service	not-found	inactive	dead	autofs.service
avahi-daemon.service	loaded	active	running	Avahi mDNS/DNS-SD Stack
blk-availability.service	loaded	inactive	dead	Availability of block devices
chronyd.service	loaded	active	running	NTP client/server

(If we want to see only active ones, we do not use **-all**).

Task 5

a) We install like that:

```
[root@rhel-9 ~ #] yum install httpd -y
```

b) We can start this service like that:

```
[root@rhel-9 ~ #] systemctl start httpd
[root@rhel-9 ~ #] systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; disabled; preset: disabled)
   Active: active (running) since Sat 2026-04-18 01:38:12 +04; 6s ago
     Docs: man:httpd.service(8)
```

c) We need to enable the service. We need to run this command:

```
[root@rhel-9 ~ #] systemctl enable httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
```

As I talked in the first task here, the soft link is created here.

d) We need to run **status** command like that:

```
[root@rhel-9 ~ #] systemctl status httpd
● httpd.service - The Apache HTTP Server
   Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
   Active: active (running) since Sat 2026-04-18 12:41:09 +04; 2min 15s ago
     Docs: man:httpd.service(8)
   Main PID: 6614 (httpd)
    Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec: 0 B/sec"
     Tasks: 177 (limit: 10435)
    Memory: 14.4M (peak: 14.7M)
       CPU: 453ms
    CGroup: /system.slice/httpd.service
            └─6614 /usr/sbin/httpd -DFOREGROUND
              └─6615 /usr/sbin/httpd -DFOREGROUND
                └─6616 /usr/sbin/httpd -DFOREGROUND
                  └─6617 /usr/sbin/httpd -DFOREGROUND
                    └─6618 /usr/sbin/httpd -DFOREGROUND

Apr 18 12:41:09 rhel-9.7 systemd[1]: Starting The Apache HTTP Server...
Apr 18 12:41:09 rhel-9.7 httpd[6614]: Server configured, listening on: port 80
Apr 18 12:41:09 rhel-9.7 systemd[1]: Started The Apache HTTP Server.
```

e) These commands just say if the service is active and enabled:

```
[root@rhel-9 ~ #] systemctl is-active httpd
active
[root@rhel-9 ~ #] systemctl is-enabled httpd
enabled
```

Task 6

We need to run `runlevel` command. The last figure in the output shows the runlevel.

```
[root@rhel-9 ~ #] runlevel
N 3
[root@rhel-9 ~ #] systemctl isolate graphical.target
[root@rhel-9 ~ #] runlevel
3 5
[root@rhel-9 ~ #]
```

As you can see, initially it was just in multi-user.target, thus the last number is 3. Then we switch to graphical.target, thus the last number here is 5.

(Btw, to be able to switch graphical.target, first we need to download GUI with `yum group install "Server with GUI"`.)

Task 7

a) I have configured my system as minimal installation. (multi-user.target). As I have shown above, first we need to create GUI tools:

```
[root@rhel-9 ~ #] yum group list
Last metadata expiration check: 1:52:26 ago on Sat 18 Apr 2026 11:00:30 AM +04.
Available Environment Groups:
  Server
  Workstation
  KDE Plasma Workspaces
  Virtualization Host
  Custom Operating System
Installed Environment Groups:
  Server with GUI
  Minimal Install
Installed Groups:
  Container Management
  Headless Management
Available Groups:
  Console Internet Tools
  .NET Development
  RPM Development Tools
  Development Tools
  Graphical Administration Tools
  Legacy UNIX Compatibility
  Network Servers
  Scientific Support
  Security Tools
  Smart Card Support
  System Tools
  Fedora Packager
  VideoLAN Client
  Xfce
[root@rhel-9 ~ #] yum group install "Server with GUI" #Install Server GUI
```

b) We can check it easily:

```
[root@rhel-9 ~ #] systemctl get-default
multi-user.target
```

c) We can run `runlevel` command again and look at the last number in its output.

```
[root@rhel-9 ~ #] runlevel
3 5
```

It does not show us target name. It shows just runlevel. We can just understand the target name from this runlevel. If it is `3`, that refers `multi-user.target` is used. However, if it is `5`, that means `graphical.target` is used.

d) We use `systemctl isolate` command to change the current target. However, if we want to change it even after the reboot process, we need to use `systemctl set-default` command:

```
[root@rhel-9 ~ #] systemctl isolate graphical.target #Just change
[root@rhel-9 ~ #] systemctl set-default graphical.target #Set default
Removed "/etc/systemd/system/default.target".
Created symlink /etc/systemd/system/default.target → /usr/lib/systemd/system/graphical.target.
```

e) `runlevel 0` is basically `poweroff.target`. When we run `init 0`, it powers the virtual machine off.

`runlevel 6` is `reboot.target`. When we run `init 6`, it just reboots the system. The same functionality with `reboot` command.

For more information:

We use `init` command to switch between runlevels. There are 7 runlevels:

- `init 0` — `poweroff.target`. When we run it, it shutdowns the system. The same with `poweroff` command.
- `init 1` — `rescue.target`. We are redirected to this runlevel if any serious problem is detected while booting. Problem in `fstab` can cause to go to this runlevel.
- `init 2` — `multi-user.target`. That is minimal installation target without network.
- `init 3` — `multiuser.target`. That is main `multi-user.target` which also has network access. That is real minimal installation.
- `init 4` — `reserve.target`. It is basically an environment to create custom target for ourselves.
- `init 5` — `graphical.target`. It is basically `multi-user.target` (runlevel 3) with Graphical User Interface.
- `init 6` — `reboot.target`. When we run it, it reboots (restarts) the system. The same with `reboot` command.

Task 8

```
[root@rhel-9 ~ #] systemctl list-units --type target --all #Including inactive ones
```

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
basic.target	loaded	active	active	Basic System
blockdev@dev-disk-by\x2duuid-2025\x2d10\x2d20\x2d23\x2d34\x2d17\x2d00.target	loaded	inactive	dead	Block Device
blockdev@dev-disk-by\x2duuid-421cc142\x2d4ad4\x2d4c88\x2dbddc\x2dd347f948c0ba.target	loaded	inactive	dead	Block Device
blockdev@dev-disk-by\x2duuid-DC82\x2d4BBF.target	loaded	inactive	dead	Block Device
blockdev@dev-mapper-rhel\x2droot.target	loaded	inactive	dead	Block Device
blockdev@dev-nvme0n1p1.target	loaded	inactive	dead	Block Device
blockdev@dev-nvme0n1p2.target	loaded	inactive	dead	Block Device
blockdev@dev-sr0.target	loaded	inactive	dead	Block Device
cryptsetup-pre.target	loaded	inactive	dead	Local Encry
cryptsetup.target	loaded	active	active	Local Encry
● display-manager.target	not-found	inactive	dead	display-man
emergency.target	loaded	inactive	dead	Emergency M
first-boot-complete.target	loaded	inactive	dead	First Boot

`--all` helps to see inactive targets, too. Without it, only active targets are shown.

To be able to see the targets which can be isolated ("AllowIsolate=yes"), we run this command:

```
[root@rhel-9 ~ #] grep -r "AllowIsolate=yes" /usr/lib/systemd/system/*.target
/usr/lib/systemd/system/ctrl-alt-del.target:AllowIsolate=yes
/usr/lib/systemd/system/default.target:AllowIsolate=yes
/usr/lib/systemd/system/emergency.target:AllowIsolate=yes
/usr/lib/systemd/system/exit.target:AllowIsolate=yes
/usr/lib/systemd/system/graphical.target:AllowIsolate=yes
/usr/lib/systemd/system/halt.target:AllowIsolate=yes
/usr/lib/systemd/system/initrd-switch-root.target:AllowIsolate=yes
/usr/lib/systemd/system/initrd.target:AllowIsolate=yes
/usr/lib/systemd/system/kexec.target:AllowIsolate=yes
/usr/lib/systemd/system/multi-user.target:AllowIsolate=yes
/usr/lib/systemd/system/poweroff.target:AllowIsolate=yes
/usr/lib/systemd/system/reboot.target:AllowIsolate=yes
/usr/lib/systemd/system/rescue.target:AllowIsolate=yes
/usr/lib/systemd/system/runlevel0.target:AllowIsolate=yes
/usr/lib/systemd/system/runlevel1.target:AllowIsolate=yes
/usr/lib/systemd/system/runlevel2.target:AllowIsolate=yes
/usr/lib/systemd/system/runlevel3.target:AllowIsolate=yes
/usr/lib/systemd/system/runlevel4.target:AllowIsolate=yes
/usr/lib/systemd/system/runlevel5.target:AllowIsolate=yes
/usr/lib/systemd/system/runlevel6.target:AllowIsolate=yes
/usr/lib/systemd/system/system-update.target:AllowIsolate=yes
```

Task 9

When we list kernel modules with `lsmod` and use `grep` command:

```
[root@rhel-9 ~ #] lsmod | grep "e1000"
e1000                200704  0
```

In my system, there is `e1000` module.

Task 10

To remove it we use `modprobe -r <MODULE\_NAME>` command:

```
[root@rhel-9 ~ #] modprobe -r "e1000"
[root@rhel-9 ~ #] lsmod | grep "e1000"
[root@rhel-9 ~ #]
```

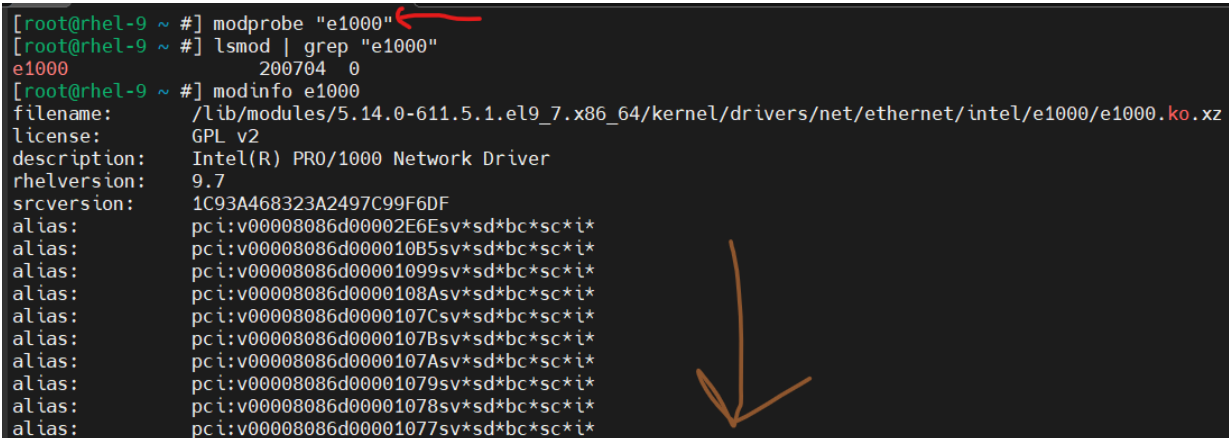
Now when we check it again, there is no `e1000` module anymore.

Task 11

To download the kernel module, we use `modprobe <MODULE_NAME>`, then we can check it with `lsmod` and `grep`.

To be able to see the module info we use `modinfo <MODULE_NAME>`.

(There is lots of info, I cut some in the image)



```
[root@rhel-9 ~ #] modprobe "e1000"
[root@rhel-9 ~ #] lsmod | grep "e1000"
e1000                200704  0
[root@rhel-9 ~ #] modinfo e1000
filename:            /lib/modules/5.14.0-611.5.1.el9_7.x86_64/kernel/drivers/net/ethernet/intel/e1000/e1000.ko.xz
license:             GPL v2
description:         Intel(R) PRO/1000 Network Driver
rhelversion:         9.7
srcversion:          1C93A468323A2497C99F6DF
alias:               pci:v00008086d00002E6Esv*sd*bc*sc*i*
alias:               pci:v00008086d000010B5sv*sd*bc*sc*i*
alias:               pci:v00008086d00001099sv*sd*bc*sc*i*
alias:               pci:v00008086d0000108Asv*sd*bc*sc*i*
alias:               pci:v00008086d0000107Csv*sd*bc*sc*i*
alias:               pci:v00008086d0000107Bsv*sd*bc*sc*i*
alias:               pci:v00008086d0000107Asv*sd*bc*sc*i*
alias:               pci:v00008086d00001079sv*sd*bc*sc*i*
alias:               pci:v00008086d00001078sv*sd*bc*sc*i*
alias:               pci:v00008086d00001077sv*sd*bc*sc*i*
```

Task 12

We can either use `dmesg` or `journalctl -dmesg` command:

```

[root@rhel-9 ~ #] dmesg
[ 0.000000] Linux version 5.14.0-611.5.1.el9_7.x86_64 (mockbuild@x86-64-03.build.eng.rdu2.redhat.com)
, GNU ld version 2.35.2-67.el9) #1 SMP PREEMPT_DYNAMIC Fri Oct 17 14:16:35 EDT 2025
[ 0.000000] The list of certified hardware and cloud instances for Red Hat Enterprise Linux 9 can be found at
//catalog.redhat.com.
[ 0.000000] Command line: BOOT_IMAGE=(hd0,gpt2)/vmlinuz-5.14.0-611.5.1.el9_7.x86_64 root=/dev/mapper:rhel--512M rd.lvm.lv=rhel/root
[ 0.000000] BIOS-provided physical RAM map:
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x0000000000000fff] ACPI NVS
[ 0.000000] BIOS-e820: [mem 0x0000000000001000-0x0000000000001fff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000002000-0x0000000000009fff] usable
[ 0.000000] BIOS-e820: [mem 0x000000000000c000-0x000000000000ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000010000-0x000000000000e03bfff] usable
[ 0.000000] BIOS-e820: [mem 0x00000000000e03c000-0x00000000000e041fff] ACPI NVS
[ 0.000000] BIOS-e820: [mem 0x00000000000e042000-0x00000000000fb6efff] usable
[ 0.000000] BIOS-e820: [mem 0x00000000000fb6f000-0x00000000000fbdefff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000000fbdf000-0x00000000000fbfafff] ACPI data
[ 0.000000] BIOS-e820: [mem 0x00000000000fbfb000-0x00000000000fbfefff] ACPI NVS
[ 0.000000] BIOS-e820: [mem 0x00000000000fbff000-0x000000000007ffffff] usable
[ 0.000000] BIOS-e820: [mem 0x00000000000ffc0000-0x00000000000ffc29fff] reserved
[ 0.000000] NX (Execute Disable) protection: active
[ 0.000000] APIC: Static calls initialized
[ 0.000000] e820: update [mem 0x0dac3018-0x0dacb057] usable ==> usable
[ 0.000000] e820: update [mem 0xdaba018-0xdabc057] usable ==> usable
[ 0.000000] e820: update [mem 0xdab7018-0xdab9057] usable ==> usable
[ 0.000000] e820: update [mem 0xdac1018-0xdac2857] usable ==> usable
[ 0.000000] e820: update [mem 0xdab5018-0xdab6e57] usable ==> usable
[ 0.000000] extended physical RAM map:
[ 0.000000] reserve setup_data: [mem 0x0000000000000000-0x0000000000000fff] ACPI NVS
[ 0.000000] reserve setup_data: [mem 0x0000000000001000-0x0000000000001fff] reserved
[ 0.000000] reserve setup_data: [mem 0x0000000000002000-0x0000000000009fff] usable
[ 0.000000] reserve setup_data: [mem 0x000000000000c000-0x000000000000ffff] reserved
[ 0.000000] reserve setup_data: [mem 0x0000000000010000-0x000000000000dab5017] usable

```

Lots of info



```

[root@rhel-9 ~ #] journalctl --dmesg
Apr 22 20:56:32 rhel-9.7 kernel: Linux version 5.14.0-611.5.1.el9_7.x86_64 (mockbuild@x86-64-03.build.eng.rdu2.redhat.com)
Apr 22 20:56:32 rhel-9.7 kernel: The list of certified hardware and cloud instances for Red Hat Enterprise Linux 9 can be found at
Apr 22 20:56:32 rhel-9.7 kernel: Command line: BOOT_IMAGE=(hd0,gpt2)/vmlinuz-5.14.0-611.5.1.el9_7.x86_64 root=/dev/mapper:rhel--512M rd.lvm.lv=rhel/root
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-provided physical RAM map:
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x0000000000000000-0x0000000000000fff] ACPI NVS
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x0000000000001000-0x0000000000001fff] reserved
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x0000000000002000-0x0000000000009fff] usable
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x000000000000c000-0x000000000000ffff] reserved
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x0000000000010000-0x000000000000e03bfff] usable
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x00000000000e03c000-0x00000000000e041fff] ACPI NVS
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x00000000000e042000-0x00000000000fb6efff] usable
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x00000000000fb6f000-0x00000000000fbdefff] reserved
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x00000000000fbdf000-0x00000000000fbfafff] ACPI data
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x00000000000fbfb000-0x00000000000fbfefff] ACPI NVS
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x00000000000fbff000-0x000000000007ffffff] usable
Apr 22 20:56:32 rhel-9.7 kernel: BIOS-e820: [mem 0x00000000000ffc0000-0x00000000000ffc29fff] reserved
Apr 22 20:56:32 rhel-9.7 kernel: NX (Execute Disable) protection: active
Apr 22 20:56:32 rhel-9.7 kernel: APIC: Static calls initialized
Apr 22 20:56:32 rhel-9.7 kernel: e820: update [mem 0x0dac3018-0x0dacb057] usable ==> usable
Apr 22 20:56:32 rhel-9.7 kernel: e820: update [mem 0xdaba018-0xdabc057] usable ==> usable
Apr 22 20:56:32 rhel-9.7 kernel: e820: update [mem 0xdab7018-0xdab9057] usable ==> usable
Apr 22 20:56:32 rhel-9.7 kernel: e820: update [mem 0xdac1018-0xdac2857] usable ==> usable
Apr 22 20:56:32 rhel-9.7 kernel: e820: update [mem 0xdab5018-0xdab6e57] usable ==> usable
Apr 22 20:56:32 rhel-9.7 kernel: extended physical RAM map:
Apr 22 20:56:32 rhel-9.7 kernel: reserve setup_data: [mem 0x0000000000000000-0x0000000000000fff] ACPI NVS
Apr 22 20:56:32 rhel-9.7 kernel: reserve setup_data: [mem 0x0000000000001000-0x0000000000001fff] reserved
Apr 22 20:56:32 rhel-9.7 kernel: reserve setup_data: [mem 0x0000000000002000-0x0000000000009fff] usable
Apr 22 20:56:32 rhel-9.7 kernel: reserve setup_data: [mem 0x000000000000c000-0x000000000000ffff] reserved
Apr 22 20:56:32 rhel-9.7 kernel: reserve setup_data: [mem 0x0000000000010000-0x000000000000dab5017] usable
Apr 22 20:56:32 rhel-9.7 kernel: reserve setup_data: [mem 0x00000000000dab5018-0x00000000000dab6e57] usable

```

Lots of info

